```
!nvidia-smi
```

```
Thu Mar 26 14:28:32 2026
+-----------------------------------------------------------------------------
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Ve
+-----------------------------------------+------------------------+------
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volat
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-U
|                                         |                        |
|=========================================+========================+======
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |
| N/A   47C    P8             11W /   70W |       0MiB /  15360MiB |
|                                         |                        |
+-----------------------------------------+------------------------+------

+-----------------------------------------------------------------------------
| Processes:
|  GPU   GI   CI              PID   Type   Process name
|        ID   ID
|=============================================================================
|  No running processes found
+-----------------------------------------------------------------------------
```

```
import numpy as np
import cupy

def FT2D_numpy(data):
  return  np.fft.ifft2(data)

def FT2D_cupy(data):
  return cupy.fft.ifft2(data)

sizes = [128,256,512,1024,2048]
for size in sizes:
  data_np = np.random.rand(size,size)
  data_cupy = cupy.random.rand(size,size)
  print(f'Numpy size {size}')
  %timeit FT2D_numpy(data_np)
  print(f'Cupy size {size}')
  %timeit FT2D_cupy(data_cupy)
```

```
Numpy size 128
245 µs ± 5.04 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
Cupy size 128
106 µs ± 15.2 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
Numpy size 256
1.29 ms ± 346 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
Cupy size 256
105 µs ± 11.5 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
Numpy size 512
4.63 ms ± 833 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
Cupy size 512
308 µs ± 236 ns per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

```
Numpy size 1024
33.1 ms ± 2.27 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
Cupy size 1024
1.3 ms ± 1.89 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
Numpy size 2048
195 ms ± 13.8 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
Cupy size 2048
5.24 ms ± 74.3 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

```
# With datatype = np.float32
sizes = [128,256,512,1024,2048]
for size in sizes:
  data_np = np.random.rand(size,size)
  data_np.astype(np.float32)
  data_cupy = cupy.random.rand(size,size)
  data_cupy.astype(cupy.float32)
  print(f'Numpy size {size}')
  %timeit FT2D_numpy(data_np)
  print(f'Cupy size {size}')
  %timeit FT2D_cupy(data_cupy)
```

```
Numpy size 128
256 µs ± 14.7 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
Cupy size 128
104 µs ± 18 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
Numpy size 256
985 µs ± 11.6 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
Cupy size 256
103 µs ± 16.2 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
Numpy size 512
4.32 ms ± 76.5 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
Cupy size 512
320 µs ± 410 ns per loop (mean ± std. dev. of 7 runs, 1000 loops each)
Numpy size 1024
36.3 ms ± 7.53 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
Cupy size 1024
1.39 ms ± 1.14 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
Numpy size 2048
186 ms ± 12.2 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
Cupy size 2048
5.52 ms ± 89.6 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

Answer:

As one expects, the larger the size, the longer it takes. At size 256x256, cupy is really starting to outperform numpy.

With datatype np.float32, I don't really notice much of a difference? The default is float64, so float32 takes up less memory, but I think there is ample memory available. Some functions and operations require float64 to avoid cancellation, but I don't think that is the issue here? It actually takes a little longer for the run to complete with float32. Perhaps some internal conversions or extra iterations needed due to the lower precision?

needed due to the lower precision?